

第 1 回 アルゴリズムと計算量

執筆: 安永憲司

1 授業の概要とねらい

コンピュータが問題を解くための手順のことをアルゴリズムと呼ぶ。効率的なアルゴリズムにはよく設計されたデータ構造が必要不可欠である。本講義ではアルゴリズムの基本的な考え方から始め、代表的なアルゴリズムとデータ構造、それらの計算量の評価、そして C 言語によるプログラムの作成について扱う。アルゴリズムは問題を解決するための具体的な手順であるため、すべての情報処理の礎であり、社会のありとあらゆるところで利用されている。アルゴリズムを学ぶことは、ものの考え方にまで影響を与えるほど重要である。

2 授業スケジュール

火曜日 5-6 時限 (13:30-15:10) に講義、火曜日 7-8 時限 (15:25-17:05) にプログラミング演習 (西 8 号館 2 階 E201)、金曜日 5-6 時限 (13:30-15:10) に講義。

- 第 1 回 (6/12) アルゴリズムと計算量：フィボナッチ数、計算量、オーダ記法、C 言語導入
- 第 2 回 (6/16) モジュール演算 1：モジュール演算、繰り返し二乗法、ユークリッドの互除法、ループ不変式
- 第 3 回 (6/16) プログラミング演習 1 (数論アルゴリズム)
- 第 4 回 (6/19) モジュール演算 2：素数性判定 (フェルマー検査)、素数生成
- 第 5 回 (6/23) データ構造 1：動的集合への操作、配列とヒープ、二分探索、連結リスト・スタック・キュー
- 第 6 回 (6/23) プログラミング演習 2 (データ構造)
- **6/26 は教員都合による休講**
- 第 7 回 (6/30) データ構造 2：二分探索木、平衡探索木、ハッシュ法
- 第 8 回 (6/30) プログラミング演習 3 (動的データ構造)
- 第 9 回 (7/3) 分割統治法 1：カラツバ法、マスター定理、マージソート、ソート計算量の下界
- 第 10 回 (7/7) 分割統治法 2：中央値、行列の掛け算 (シュトラッセン法)
- 第 11 回 (7/7) プログラミング演習 4 (データ構造と分割統治法)
- 第 12 回 (7/10) 高速フーリエ変換 1：多項式の係数表現と値表現、多項式評価と補間、ヴァンデルモンド行列
- 第 13 回 (7/14) 高速フーリエ変換 2：高速フーリエ変換 (FFT)、FFT の応用
- 第 14 回 (7/14) プログラミング演習 5 (高速フーリエ変換)
- 第 15 回 (7/17) グラフアルゴリズム 1：グラフの表現、深さ優先探索、到達可能性、連結成分
- 第 16 回 (7/21) グラフアルゴリズム 2：幅優先探索、最短パス、ダイクストラ法

- 第 17 回 (7/21) プログラミング演習 6 (グラフアルゴリズム)
- 第 18 回 (7/24) 動的計画法 1: 最短パス, 最長増加部分列, 編集距離
- 第 19 回 (7/28) 動的計画法 2: ナップサック問題, 連鎖行列積
- 第 20 回 (7/28) プログラミング演習 7 (動的計画法)
- 第 21 回 (7/31) 発展的内容 (量子アルゴリズム): 量子ビット, 量子フーリエ変換, ショアのアルゴリズム

3 授業評価

- 講義時間内の小テスト (毎火曜日に実施) 計 7 回のうち最悪点の 1 回を除いた 6 回分で評価 (30%)
- 宿題形式のレポート 計 2 回 (30%)
- プログラミング演習課題 計 7 回 (40%)

4 なぜアルゴリズムとデータ構造を学ぶのか

- **重要性:** コンピュータサイエンスやその関連分野の様々な場面でアルゴリズムやデータ構造が登場し, それらが実社会で使われている. 例えば, 通信におけるルーティングプロトコル, セキュア通信のための暗号化の計算, コンピュータグラフィックスのためのアルゴリズム, 各種データベースを実現するためのデータ構造, 様々なデータを解析するためのグラフアルゴリズム. その基本を理解することは, 世の中の仕組みを理解することにもつながる.
- **技術革新:** アルゴリズムの設計は技術革新につながる. 検索エンジンは代表的な例であり, 機械学習の発展においてもアルゴリズムは貢献している. よいアルゴリズムが見つからない問題を活用することで公開鍵暗号が実現されている.
- **新しい視点:** アルゴリズムやデータ構造を理解することは, コンピュータサイエンス以外の分野において新たな「視点」を与えることにつながる. 量子計算・量子アルゴリズムの登場は, 量子力学に計算の観点を与えている. 市場における価格変動はアルゴリズムの計算過程とみなすこともできる.

5 フィボナッチ数

フィボナッチ数とは,

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, \dots$$

と続く数列であり,

$$F_n = \begin{cases} F_{n-1} + F_{n-2} & n > 1 \\ 1 & n = 1 \\ 0 & n = 0 \end{cases}$$

という漸化式で与えられる. 実際の値は,

$$F_{20} = 6765, F_{40} = 102334155, F_{60} \approx 1.55 \times 10^{12}, F_{80} \approx 2.34 \times 10^{16}, F_{100} \approx 3.54 \times 10^{20}$$

のように大きくなり, 一般に,

$$F_n = \frac{1}{\sqrt{5}} \left(\frac{1+\sqrt{5}}{2} \right)^n - \frac{1}{\sqrt{5}} \left(\frac{1-\sqrt{5}}{2} \right)^n \approx 2^{0.694n}$$

であり、 n について指数関数的に大きな値をとる。入力として n が与えられたときに、 F_n の値を答えるアルゴリズムを考えてみよう。

指数時間アルゴリズム

漸化式に従えば、以下のアルゴリズム（C 言語プログラムの関数として記述）で計算できる。

```
1  int fib1 ( int n ) {
2      if ( n == 0 ) {
3          return 0;
4      } else if ( n == 1 ) {
5          return 1;
6      } else {
7          return fib1( n-1 ) + fib1( n-2 );
8      }
9  }
```

このアルゴリズムは、`fib1` という関数の計算の内部で、`fib1(n-1)` のように自分自身を呼び出している。このようなアルゴリズムは**再帰アルゴリズム** (*recursive algorithm*) と呼ばれ、アルゴリズムの設計で度々登場する。

アルゴリズムの計算量 $T(n)$ を見積もりたい。計算量は、様々な観点から定めることができるが、実行時間 (running time) に近い指標として、「計算ステップ数」を考える (`fib1` の場合、整数値の比較と足し算の回数)。答え F_n を出力するための計算量が $T(n)$ のとき、

$$T(n) = T(n-1) + T(n-2) + 3$$

が成り立つ。明らかに $T(n) \geq F_n$ が成り立つため、 $T(n)$ は n の指数関数である。

多項式時間アルゴリズム

`fib1` の計算量が多い理由は、同じ計算を繰り返し行っているためである。`fib1(n)` の計算のため、`fib1(n-1)` と `fib1(n-2)` を呼び出すが、これらはそれぞれ、`fib1(n-2)` と `fib1(n-3)`、`fib1(n-3)` と `fib1(n-4)` を呼び出す。つまり、`fib1(n-2)` や `fib1(n-3)` を何度も呼び出しており、明らかに無駄な計算をしている。

無駄をなくすため、`fib1(n-2)` や `fib1(n-3)` など一度計算したあと、その値を覚えておき（**配列** (*array*) に格納する）、二度目以降は格納された値を読み込むようにすればよい。以下はそのアルゴリズムである。

```
1  int fib2 ( int n ) {
2      int i;
3      if ( n == 0 ) {
4          return 0;
5      }
6      // 配列 f[0] から f[n] を準備（その実現方法は省略）
7      f[0] = 0;      // f[0] に 0 を格納
8      f[1] = 1;      // f[1] に 1 を格納
9      for ( i = 2; i < n+1; i++ ) {
10         f[i] = f[i-1] + f[i-2];      // f[i] に f[i-1] + f[i-2] を格納
11     }
```

```

12     return f[n];
13 }

```

このアルゴリズムの計算量を見積もると、for ループで繰り返される部分は n に比例する程度の計算量であり、 n について線形の計算量である。

少し注意が必要なのは、 F_n という整数値自体が $2^{0.694n}$ 程度の大きさであり、 n に比例する長さのビット数で表す必要がある。 n ビットの数同士の足し算の計算量は n に比例するため、fib2 内の足し算の計算量を考慮すると、fib2 の計算量は n^2 に比例する程度である。

6 アルゴリズムの計算量

6.1 オーダ記法・漸近記法

fib2 の計算量を正確に求めるには、配列 f を準備する計算量、for ループの $i++$ の計算量、配列 $f[i-1]$ にアクセスする計算量など、すべての部分を正確に求める必要がある。このようにして計算量を正確に導出することも可能であるが、適度に簡略化して計算量を見積もりたい。実際、一つの計算ステップの「実行時間」は、それを実行するコンピュータの実現方法に大きく依存し、正確に求めることはそもそも難しい。

計算量を適度に簡略化して表現する方法として、**オーダ記法・ビッグオー記法** (*big-O notation*)・**漸近記法** (*asymptotic notation*) などと呼ばれるものが便利である。正確な計算量が $2n^2 + 3n + 4$ であるとき、それを $O(n^2)$ と表す方法である。主要項である n^2 だけを残し、その係数も省略している。計算量が $O(n^2)$ であるとは、「計算量は高々 n^2 に比例する程度である」と解釈できる。厳密には、以下の通り定義できる。

関数 $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ について、定数 $c > 0$ と $n_0 \in \mathbb{N}$ が存在し、任意の $n \geq n_0$ に対し、 $f(n) \leq c \cdot g(n)$ であるとき、 $f(n) = O(g(n))$ と書く。

直観的には、定数倍を無視して「 $f \leq g$ 」と表したいときが $f(n) = O(g(n))$ である。定義より、 $O(\cdot)$ は n が十分大きいときの関数の振る舞いを見ている。そのため、**漸近的な** (*asymptotic*) **上界**と言われる。

クイズ 6.1. 以下のうち、正しいものをすべて選べ。

- | | |
|-------------------------------|--------------------------------------|
| 1. $5n^4 + 3n^2 + 1 = O(n^5)$ | 4. $\frac{n}{\log_2 n} = O(n^{0.9})$ |
| 2. $n^3 + 2n^2 + 10 = O(n^4)$ | 5. $2^{n+10} = O(2^n)$ |
| 3. $100 = O(1)$ | 6. $4^n = O(2^n)$ |

6.2 アルゴリズム解析の原則

アルゴリズムの計算量・実行時間を見積もる際の原則を述べる。

- 最悪時解析** (*worst-case analysis*): アルゴリズムは、与えられた入力に対して所望の計算結果を返すための手続きであり、計算ステップ数は入力に依存して決まる。通常は、入力サイズ n の任意の入力が与えられたときの最大の（最悪の）実行時間を考え、それを n の関数として表したものがそのアルゴリズムの実行時間である。
- 漸近的解析** (*asymptotic analysis*): 計算量・実行時間は、増加の程度 (order of growth) だけに着目し、主要項だけを見る。 $5n + 50$ と $n^2 + 10$ を比べると、小さい n では前者の方が大きいですが、 n が大きくなると n^2 の影響が大きくなり、後者の方が大きくなる (図 1)。この場合、 $O(n)$ と $O(n^2)$ なの

で後者の計算量が大きいと考える．定数項を無視することで，議論が簡素になる．

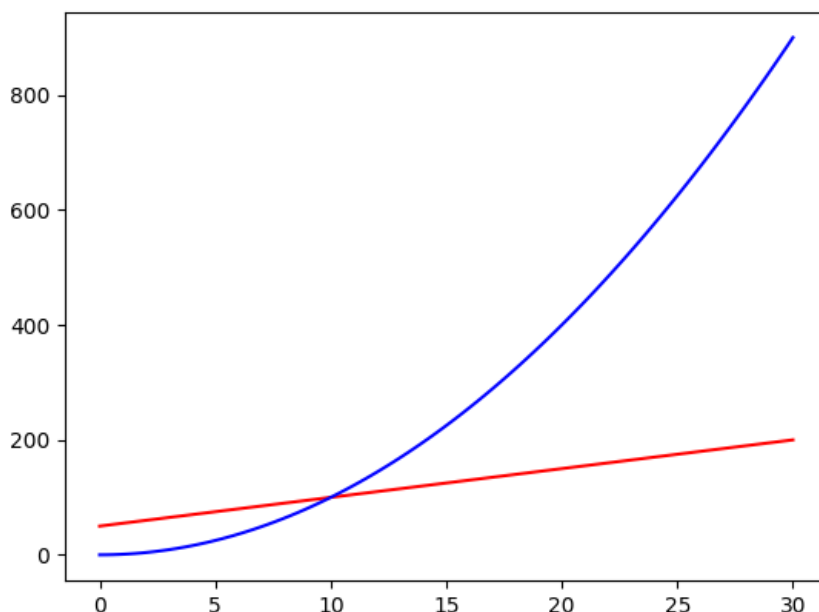


図 1 関数 $5n + 50$ (赤色) と $n^2 + 10$ (青色)

6.3 その他の漸近記法 $\Theta(\cdot)$, $\Omega(\cdot)$, $o(\cdot)$, $\omega(\cdot)$

フィボナッチ数の多項式時間アルゴリズムに対して，実行時間は $O(n^2)$ であるという説明をしたが， $O(\cdot)$ の定義に従えば， $O(n^3)$ だと主張しても問題ない．しかし，これは少し奇妙な主張であり，見積もりが甘い．そこで，計算量について，より緊密な評価を行いたいときに登場するのが $\Theta(\cdot)$ である．定義は以下の通りである．

関数 $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ について，定数 $c_1, c_2 > 0$ と $n_0 \in \mathbb{N}$ が存在し，任意の $n \geq n_0$ に対し， $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ であるとき， $f(n) = \Theta(g(n))$ と書く．

Θ 記法を用いれば，フィボナッチ数の多項式アルゴリズムを $\Theta(n^3)$ と主張することはできない．

アルゴリズムが定まれば，その実行時間を導出することができ，原理的には Θ 記法によって実行時間を与えることができる．しかし，場合によっては厳密な計算量の評価が難しい場合や，複雑になる場合もあり（フィボナッチ数の指数時間アルゴリズムの実行時間を $\Theta(\cdot)$ で与えてみよう），上界さえ与えれば十分なことが多い．そのため， $O(\cdot)$ による評価が標準的である．

一方で，漸近的な下界に対応するのが Ω 記法である．

関数 $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ について，定数 $c > 0$ と $n_0 \in \mathbb{N}$ が存在し，任意の $n \geq n_0$ に対し， $c \cdot g(n) \leq f(n)$ であるとき， $f(n) = \Omega(g(n))$ と書く．

定数倍や低次項を無視した漸近的な意味で，「 $f \leq g$ 」，「 $f = g$ 」，「 $f \geq g$ 」に対応するのがそれぞれ $f(n) = O(g(n))$ ， $f(n) = \Theta(g(n))$ ， $f(n) = \Omega(g(n))$ である．この他に，「 $f < g$ 」や「 $f > g$ 」に対応する記法も存在する．

関数 $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ について、任意の定数 $c > 0$ に対し、 $n_0 \in \mathbb{N}$ が存在し、任意の $n \geq n_0$ に対し、 $f(n) < c \cdot g(n)$ であるとき、 $f(n) = o(g(n))$ と書く。

関数 $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ について、任意の定数 $c > 0$ に対し、 $n_0 \in \mathbb{N}$ が存在し、任意の $n \geq n_0$ に対し、 $f(n) > c \cdot g(n)$ であるとき、 $f(n) = \omega(g(n))$ と書く。

6.4 漸近記法に関する補足

これまでの漸近記法は等号の右側にしか登場しなかったが、左側にも登場し、

$$2n^2 + 3n + 4 = 2n^2 + \Theta(n) = \Theta(n^2)$$

などのように評価する場合がある。このときは、左側に登場する漸近記法としてどの関数が選択されたとしても、右側に登場する漸近記法の関数を適切に選択することで、等号を成立させることができると解釈すれば良い。上記の例では、最初の $\Theta(n)$ には、 $3n, 4n + 5, 6n + 100 + 1/n$ などの関数が考えられるが漸近記法のため定まっておらず、どの関数が選ばれても、右側の $\Theta(n^2)$ として適切な関数が存在する。例えば、 $\Theta(n)$ として $3n$ が選ばれれば、 $\Theta(n^2)$ は $2n^2 + 3n$ とすることで等号が成立する。

漸近記法を用いるときは、どの変数が大きくなる場合を考えているのか、文脈から判断する必要がある。 $O(f(n))$ であれば n が大きくなる場合であり、 $O(g(m))$ ならば m が大きくなる場合である。 $f(n) = O(1)$ と表せば、 n が大きくなる場合を考えているはずである。

アルゴリズム `fib2(n)` の計算量 $T(n)$ に対し、「 $n < 3$ のときは $T(n) = O(1)$ 」と書くことがある。 $O(\cdot)$ の定義に従えば、 n が十分に大きいときにある定数で上から抑えられることを表す記法であり、 $n < 3$ の場合に定数で抑えられる必要はまったくなく、意味のない主張に見える。しかし、慣例として、 $n < 3$ の範囲では $T(n) \leq c$ を満たす定数 $c > 0$ が存在すると解釈でき、そうであれば意味のある主張である。このような誤用 (abuse) を許容することで、 $O(1)$ に隠れた定数に言及する必要がなくなり、解析においてより重要な変数に集中することができる。

6.5 計算量に関するクイズ・演習問題

クイズ 6.2. n 個の整数を格納した配列 A と整数 t を入力とし、配列 A の中に値 t が格納されているか否かを返す以下のアルゴリズムを考える。

```
1  for ( i = 0; i < n; i++ ) {
2      if ( A[i] == t ) return 1;
3  }
4  return 0;
```

このアルゴリズムについて正しいものをすべて選べ。

1. 最良時の実行時間は $O(1)$ である。
2. 最悪時の実行時間は $O(n)$ である。
3. 最悪時の実行時間は $O(n^2)$ でもある。
4. 最悪時の実行時間は $\Theta(n^2)$ である。

クイズ 6.3. 次のアルゴリズムの実行時間として最も適切なものを選び.

```
1  for ( i = 0; i < n; i++ ) {  
2      if ( A[i] == t ) return 1;  
3  }  
4  for ( i = 0; i < n; i++ ) {  
5      if ( B[i] == t ) return 1;  
6  }  
7  return 0;
```

1. $O(1)$ 2. $O(\log_2 n)$ 3. $O(n)$ 4. $O(n^2)$

クイズ 6.4. 次のアルゴリズムの実行時間として最も適切なものを選び.

```
1  for ( i = 0; i < n; i++ ) {  
2      for ( j = 0; j < n; j++ ) {  
3          if ( A[i] == B[j] ) return 1;  
4      }  
5  }  
6  return 0;
```

1. $O(1)$ 2. $O(\log_2 n)$ 3. $O(n)$ 4. $O(n^2)$

クイズ 6.5. 次のアルゴリズムの実行時間として最も適切なものを選び.

```
1  for ( i = 0; i < n; i++ ) {  
2      for ( j = i+1; j < n; j++ ) {  
3          if ( A[i] == A[j] ) return 1;  
4      }  
5  }  
6  return 0;
```

1. $O(1)$ 2. $O(\log_2 n)$ 3. $O(n)$ 4. $O(n^2)$

演習問題 6.1. 関数 $f, g: \mathbb{N} \rightarrow \mathbb{R}^+$ に関して以下を示せ.

1. $f(n) = O(g(n))$ と $g(n) = \Omega(f(n))$ は等価である.
2. $f(n) = \Theta(g(n))$ と $f(n) = O(g(n))$ かつ $f(n) = \Omega(g(n))$ は等価である.
3. $f(n) = O(g(n))$ と $\limsup_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty$ は等価である.
4. $f(n) = \Omega(g(n))$ と $\liminf_{n \rightarrow \infty} \frac{f(n)}{g(n)} > 0$ は等価である.
5. $f(n) = o(g(n))$ と $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$ は等価である.
6. $f(n) = \omega(g(n))$ と $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$ は等価である.

演習問題 6.2. 以下の主張が正しいかどうかを判定せよ. 正しい場合は証明し, 正しくない場合は反例を与えよ.

1. $f(n) = O(g(n))$ かつ $g(n) = O(h(n))$ ならば $f(n) = O(h(n))$ である.
2. $f(n) = O(g(n))$ かつ $g(n) = O(f(n))$ ならば $f(n) = \Theta(g(n))$ である.
3. $f(n) = O(g(n))$ ならば $2^{f(n)} = O(2^{g(n)})$ である.

4. $f(n) = O(g(n))$ かつ $f(n), g(n) \geq 2$ ならば $\log_2 f(n) = O(\log_2 g(n))$ である.

演習問題 6.3. 正の実数 $c > 0$ に対し $f(n) = 1 + c + c^2 + \cdots + c^n$ と定めるとき, 以下を示せ.

1. $c < 1$ ならば $f(n) = \Theta(1)$ である.
2. $c = 1$ ならば $f(n) = \Theta(n)$ である.
3. $c > 1$ ならば $f(n) = \Theta(c^n)$ である.

演習問題 6.4. 「アルゴリズム A の実行時間は少なくとも $O(n^2)$ である」という主張が無意味であることを説明せよ.

演習問題 6.5. 以下を示せ.

1. $n! = o(n^n)$
2. $n! = \omega(2^n)$
3. $\log_2(n!) = \Theta(n \log_2 n)$

演習問題 6.6. 関数 (a) から (e) を漸近的に小さい順に並べよ. ただし, $f(n)$ を $g(n)$ より先に書くとは, $f(n) = O(g(n))$ であることを意味する.

1. (a) $\sqrt{n}$ (b) 10^n (c) $n^{1.5}$ (d) $2^{\sqrt{\log_2 n}}$ (e) $n^{5/3}$
2. (a) $n^2 \log_2 n$ (b) 2^n (c) 2^{2^n} (d) $n^{\log_2 n}$ (e) n^2
3. (a) $2^{\log_2 n}$ (b) $2^{2 \log_2 n}$ (c) $n^{5/2}$ (d) 2^{n^2} (e) $n^2 \log_2 n$
4. (a) $(\log_2 n)^3$ (b) $2^{\sqrt{\log_2 n}}$ (c) $n^{0.01}$ (d) $\frac{n}{\log_2 n}$ (e) n

演習問題 6.7. 関数 $f, g : \mathbb{N} \rightarrow \mathbb{R}^+$ を

$$f(n) = n^{1+(-1)^n}, \quad g(n) = n$$

と定める. 以下のうち成り立つものをすべて答え, 理由を述べよ.

$$f(n) = O(g(n)), \quad f(n) = \Omega(g(n)), \quad f(n) = \Theta(g(n)), \quad f(n) = o(g(n)), \quad f(n) = \omega(g(n)).$$

クイズの答え

クイズ 6.1

正しいものは 1, 2, 3, 5 である。1 と 2 は上界として正しい。3 は定数時間である。5 は $2^{n+10} = 2^{10} \cdot 2^n$ であり、定数倍の違いなので $O(2^n)$ である。一方、4 は正しくない。また、 $4^n = 2^{2n}$ は 2^n の定数倍では抑えられないため、6 も正しくない。

クイズ 6.2

正しいものは 1, 2, 3 である。最良時には最初の要素が t に等しければすぐに終了するので $O(1)$ である。最悪時には配列全体を調べるため $O(n)$ である。また、 $O(n)$ ならば $O(n^2)$ でもある。ただし、最悪時実行時間は $\Theta(n)$ であり、 $\Theta(n^2)$ ではない。

クイズ 6.3

答えは 3 である。2 つの for ループは順に実行されるので、実行時間は $O(n) + O(n) = O(n)$ である。

クイズ 6.4

答えは 4 である。二重ループであり、最悪時には n^2 回程度の比較が行われるため、実行時間は $O(n^2)$ である。

クイズ 6.5

答えは 4 である。比較回数は

$$(n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}$$

であり、これは $\Theta(n^2)$ である。